CSA37 – SOFTWARE TESTING

UNIT – 01 ASSIGNMENT

Assignment Title : Evaluate the Quality Process and Defect Profile of a Hospital Appointment Application

Submitted By
Nadhiya SS - 192321136

Executive Summary and Contents

MediQueue serves about 6,000 patients a day, yet the last release was built by five developers who tested their own code, and was approved because a demo "showed no errors". Three months later patients found 60 of the 100 defects. This report investigates why, and recommends how quality should be built into the process. All figures come from the defect-data table in the assignment; every assumption I had to add is flagged in the text and listed in Appendix A.

Key findings at a glance

Finding	Evidence
Testing was not an engineering activity	No plan, no measurement, no standards, not repeatable; approval depended on a demo (Task 1)
Quality failed because of process, not only people	Only 38 of 100 defects originate in coding; 62 originate elsewhere. 5 of 10 incidents could not be prevented by a better developer (Task 2)
Late detection is extremely expensive	Total fixing cost ₹26,10,500; production defects were 60% of defects but 91.9% of the cost; only 40% of defects were removed before release (Task 3)
Reviews would have paid for themselves	Requirement + design reviews: estimated saving ₹4.3–8.8 lakh against about ₹0.5 lakh of review effort (Task 3)
Defects start upstream	5 of 10 incidents originate in requirements or design; 52% of introduced defects are pre-coding (Task 7)
Priority class to attack first	Requirements / specification defects, with an immediate hotfix for the patient-safety defect I6 (Task 8)
Recommended plan	Defect repository, glossary, requirement and design reviews, independent tester, peer code review. Expected: production defects 60 → about 24; cost ₹26.1 → ₹12.4 lakh (Task 11)
Verdict for the new conditions	MODIFY the plan: keep the foundations; make reviews lightweight, automate regression, add contract testing and external-defect tracking (Task 12)

Contents

Task	Title	Marks
Task 1	Evaluate Testing as an Engineering Activity	5
Task 2	Evaluate the Role of Process in Software Quality	2
Task 3	Evaluate Testing as a Process (with cost calculations)	6
Task 4	Apply and Critique Basic Definitions	4
Task 5	Evaluate the Application of Testing Principles	3
Task 6	Evaluate the Tester's Role	3
Task 7	Trace the Origins of Defects	4
Task 8	Evaluate Defect Classes and Severity	4
Task 9	Design and Appraise a Defect Repository	3
Task 10	Evaluate Defect Examples and Prevention Strategies	4
Task 11	Recommend and Defend a Quality Improvement Plan	4
Task 12	Day-4 Design Challenge: Re-evaluate and Adapt the Quality Process (A–D)	8
Appendix	Assumptions, limitations and references	–
	TOTAL	**50**

Task 1 – Evaluate Testing as an Engineering Activity (5 marks)

Position: MediQueue's testing cannot be called an engineering activity. The team did not engineer quality into the product; they hoped for it, and the hospital's approval rule ("the demo showed no errors") rewarded hope.

1.1 Characteristics of testing as an engineering activity

Engineering means achieving a goal in a planned, measured, standardised and repeatable way, so that results do not depend on luck or on one person's memory. For testing this gives four core characteristics, plus two that support them: defined roles (independence and competence) and feedback for improvement.

1.2 Judging MediQueue against each characteristic

Characteristic	What engineering practice requires	What MediQueue did (evidence)	Incidents it contributed to
Planning	Test policy, test plan, objectives, risks, schedule, resources, entry/exit criteria	No plan. Developers "tested" their own code "just before release"; there was no time or owner for testing	I2, I10: nobody planned to test business rules or performance
Measurement	Coverage, defects found per phase, removal efficiency, cost per defect, exit criteria	Only measure was "no errors in the demo". Retrospectively only 40 of 100 defects (40%) were found before release	I10: no response-time target; whole profile of 60 production defects
Standards and techniques	Documented test design techniques (equivalence classes, boundary values), reviewed test cases with expected results	Demo-based testing; no test design technique; test cases not reviewed	I4 (40-character boundary), I5 (age 60 boundary), I8 (wrong expected value)
Repeatability	Documented, re-runnable (ideally automated) tests, controlled test environment and data	A demo cannot be repeated exactly; no regression pack; intermittent defects (same-second timing) were never provoked	I1, I6 (timing / concurrency), I9 (environment change)
Defined roles and independence	Trained testers, review by someone other than the author	No separate tester; authors judged their own work	I3, I8
Feedback and improvement	Defect repository, root-cause analysis, lessons learned	Complaints held in e-mail and a spreadsheet; no analysis of where defects come from	All: the same classes of defects will repeat

1.3 How the absence of each characteristic produced the incidents

- **No planning** meant testing was whatever time remained. Rules never written down (Sunday closure, I2) and non-functional goals (25-second schedule screen, I10) were never put on any list of things to verify.
- **No measurement** meant management could not see risk. A demo with "no errors" gave a false sense of safety; the real defect removal efficiency before release was only 40%.
- **No standards or techniques** meant simple boundary cases were missed: a name longer than 40 characters (I4) and a patient turning 60 on the appointment day (I5) are textbook boundary-value tests that cost minutes to design.
- **No repeatability** meant the intermittent defects (I1, I6) were invisible: they appear only when two actions occur at the same instant, which a one-off demo never does but 6,000 daily users do.
- **No independence** meant the person who misunderstood the GST rule (I3) was also the person who judged it correct, and a test case with a wrong expected value (I8) was never reviewed by a second person.

1.4 What an engineered testing approach would have looked like

Level / activity	Engineered practice	Technique	Incidents it would expose
Test planning (start of project)	Test policy and plan; risk list (patient safety, money, privacy, availability); entry/exit criteria such as "no open Critical or High defect"	Risk-based prioritisation	All, by directing effort to high-risk areas
Requirement review	Inspect requirements with hospital staff; derive acceptance tests from them	Checklist-based inspection	I2, I3, I10
Unit testing	Automated tests for each module, including invalid and boundary input	Equivalence partitioning, boundary value analysis	I4, I5
Integration and system testing	Concurrency, payment-consistency and performance tests with reviewed expected results	Parallel-request tests, load test, test oracles from the spec	I1, I3, I6, I10
Documentation check	Walk the user manual against the running application	Scenario walkthrough	I7
Regression and monitoring	Automated regression pack; alerts when SMS fails	CI pipeline; monitoring	I9, recurrence of any defect

Verdict. On every characteristic MediQueue scores "absent" or "weak". The project had effort but no engineering: nothing was planned, measured, standardised or repeatable, so the 60 production defects were a predictable result and not bad luck.

Task 2 – Evaluate the Role of Process in Software Quality (2 marks)

> **Position:** Quality is mainly the result of a good process, operated by competent people. Hiring better developers alone would not have prevented most incidents.

2.1 Role of a defined process

A defined process makes quality independent of individuals: it says what is checked, by whom, when and against what standard, and it creates feedback so mistakes are found where they are cheap to fix. Skilled people reduce the number of mistakes they make, but they cannot see their own blind spots (the same mind that misunderstands a rule will also "confirm" it in a self-check).

2.2 Would better developers alone have prevented the incidents?

Incident	Cause	Prevented by better developers alone?	Reason
I1	Concurrent token allocation	Partly	Skilled developers know about race conditions, but without a design review or a concurrency test even they cannot prove the code is safe
I2	Sunday rule never written	**No**	Nobody can code a rule that was never stated
I3	GST rule misunderstood	**No**	A domain misunderstanding; needs requirement review with the finance team
I4	Crash on name > 40 characters	**Yes**	A coding and validation slip a stronger developer would likely avoid
I5	Age 60 on the appointment day	Partly	Experienced developers handle boundaries, but the rule was probably not precisely specified
I6	Shared state between two doctors	Partly	Concurrency again; needs review and testing as well as skill
I7	Manual describes a missing menu	**No**	Documentation was not kept consistent with the product; a process gap
I8	Wrong expected value, "Pass"	**No**	Test-process failure: no independent review of test cases
I9	SMS key change stops reminders	**No**	Operational and design process: no monitoring or failure handling for external dependencies
I10	25-second schedule screen	Partly	No performance requirement or test existed, so skill had no target

Score: only **1 of 10** incidents is clearly preventable by developer skill alone, **4** partly, and **5** not at all. The defect data agree: coding accounts for 38 of 100 introduced defects, so even a team that never made a coding mistake would still have shipped 62 defects from requirements (30), design (22), testing (5) and documentation (5).

2.3 Process maturity view (TMM)

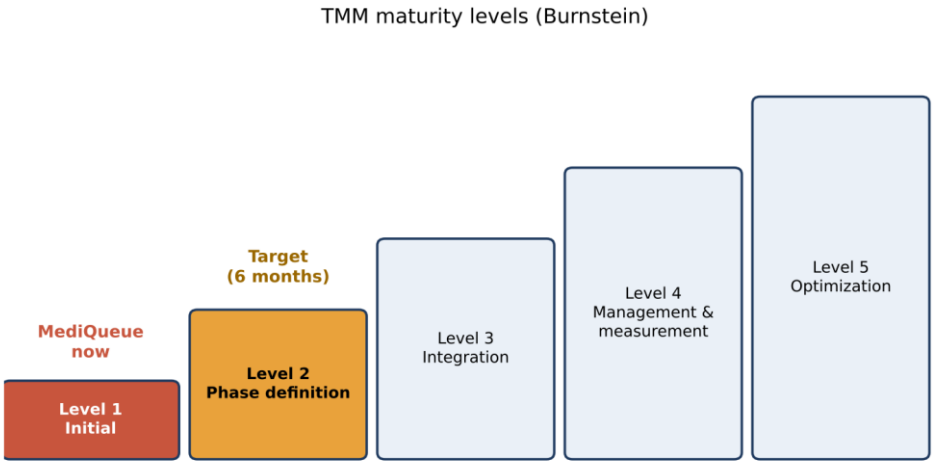


Figure 1: Testing Maturity Model levels and the position of MediQueue

MediQueue sits at **TMM Level 1 (Initial)**. At this level testing is not distinguished from debugging, there are no testing goals, tests are ad hoc and written after the code, and the aim of testing is to show that the software runs. MediQueue matches each point: developers tested their own code just before release, and approval was based on a demo. The realistic six-month target is **Level 2 (Phase Definition)**: separate testing goals from debugging goals, introduce test planning, and institutionalise basic techniques such as boundary value analysis. Levels 3 to 5 (test organisation, peer reviews and measurement, defect prevention) come later.

Conclusion: Better developers are worth having, but they raise the ceiling of what a process can achieve; they do not replace it. Management's plan would address about a third of the problem.

Task 3 – Evaluate Testing as a Process (6 marks)

Position: Treated as a process running through the whole life cycle, testing would have moved most detections to phases where fixes cost ₹500–₹3,000 instead of ₹40,000. My estimate is that this saves between ₹4.3 and ₹8.8 lakh per release (16%–34% of the ₹26.1 lakh spent), at a review cost of about ₹0.5 lakh.

3.1 Testing as a process

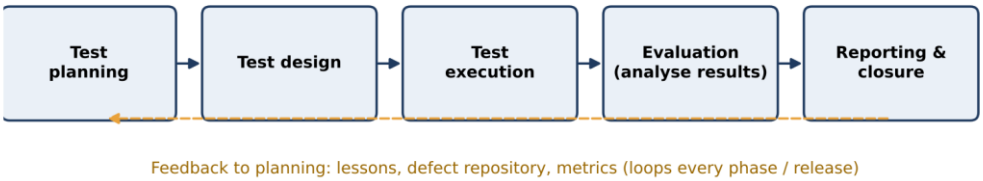


Figure 2: Testing process activities with a feedback loop

A testing process has five activities: **planning** (objectives, scope, risks, resources), **design** (test cases, expected results, test data), **execution** (running tests in a controlled environment), **evaluation** (comparing actual and expected results, deciding pass or fail, analysing defects) and **reporting** (defect and metric reports, exit decision). These activities repeat at every development phase and feed lessons back into planning. MediQueue had only a thin, informal version of "execution", with no planning, design, evaluation or reporting.

3.2 Verification and validation in each phase

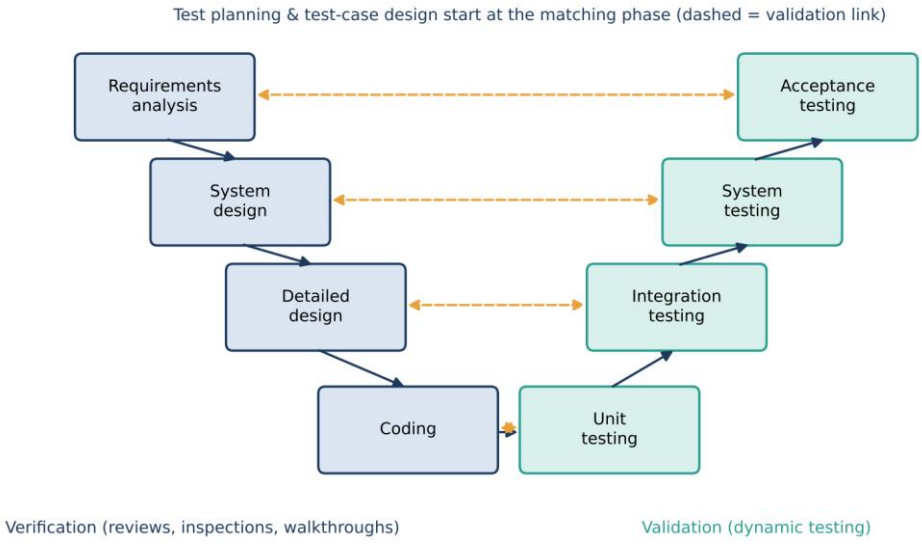


Figure 3: V-model: each development phase paired with a test level

Phase	Verification (are we building it right?)	Validation / test level	Test-planning work in this phase
Requirements	Requirement inspection, traceability check, glossary	Acceptance testing against business needs	Write acceptance criteria and test oracles
Design	Design inspection and walkthrough (concurrency, failure handling)	System and integration testing	Plan system and integration tests
Coding	Peer code review, static analysis, coding standard	Unit testing	Design unit tests with boundary values

Phase	Verification (are we building it right?)	Validation / test level	Test-planning work in this phase
Testing	Review of test cases and expected results	System, regression, performance tests	Plan regression and exit criteria
Documentation	Review of manual and help against the product	Scenario walkthrough of the manual	Include documentation in the exit checklist
Release / operation	Release readiness review	Monitoring, regression after each release	Plan monitoring and defect feedback

3.3 Phase in which each MediQueue defect should have been detected

Incident	Defect	Phase where it should have been detected	Detecting activity
I1	Duplicate token number	Design (then integration test)	Design inspection for concurrency; parallel-booking test
I2	Sunday appointments accepted	Requirements	Requirement review ("which days is OPD open?")
I3	₹500 shown, ₹550 charged	Requirements (then system test)	Requirement review with finance; price-consistency test
I4	Crash on long name	Coding (unit test)	Boundary value test at 40 and 41 characters
I5	Age-60 priority missed	Coding (unit test)	Boundary value test on the date of birth
I6	Wrong prescription details	Coding / integration test	Code review of shared state; concurrent-save test
I7	Manual describes missing menu	Documentation	Walkthrough of manual vs. application
I8	Wrong expected value, "Pass"	Testing	Independent review of test cases
I9	SMS fails after key change	Design (then integration)	Design review of external dependency; failure test; monitoring
I10	25-second load	Requirements (then system test)	Performance requirement; load test

3.4 What the hospital paid because defects were found late

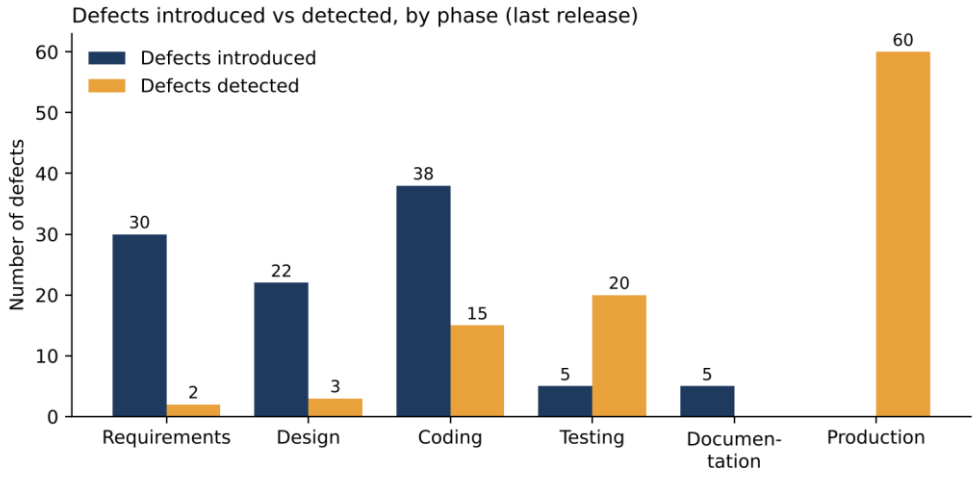


Figure 3a: Defects introduced vs detected in each phase (last release)

Table 3.1 – Actual cost of fixing defects (defects detected × cost per defect)

Phase detected	Defects detected	Cost per defect (₹)	Calculation	Total cost (₹)
Requirements	2	500	2 × 500	1,000
Design	3	1,500	3 × 1,500	4,500

Phase detected	Defects detected	Cost per defect (₹)	Calculation	Total cost (₹)
Coding	15	3,000	15 × 3,000	45,000
Testing	20	8,000	20 × 8,000	1,60,000
Documentation	0	–	–	0
Production	60	40,000	60 × 40,000	24,00,000
Total	**100**			**26,10,500**

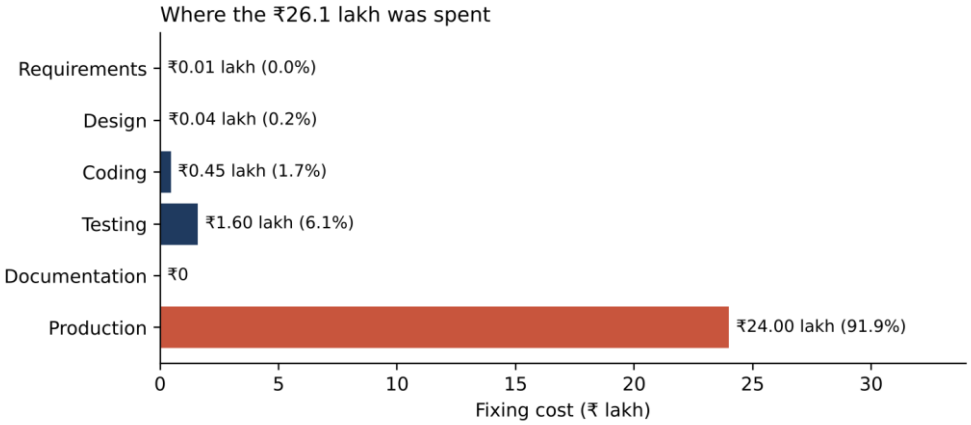


Figure 4: Fixing cost by phase of detection

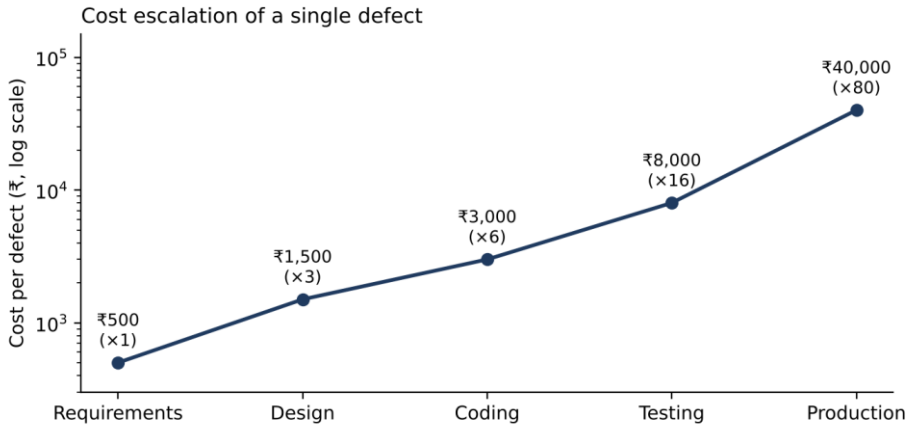


Figure 5: Cost escalation of one defect (log scale)

Interpretation.

- Production defects were **60% of all defects** but **91.9% of the cost** (24,00,000 ÷ 26,10,500). Requirements and design detections together were 5 defects and cost ₹5,500, or 0.2%.
- A production fix costs **80 times** a requirements fix (40,000 ÷ 500), 27 times a design fix and 13 times a coding fix. The last row of the table is the price of the missing process.
- **Defect removal efficiency before release** = (2 + 3 + 15 + 20) ÷ 100 = **40%**. Six of every ten defects reached patients.
- If every defect had been found in the phase where it was introduced (30 × 500 + 22 × 1,500 + 38 × 3,000 + 5 × 8,000 + 5 × 1,000) the cost would have been about **₹2,07,000** (I assumed ₹1,000 for a documentation fix, since the case gives none). The hospital paid ₹26,10,500, i.e. **₹24,03,500 more (about 12.6 times)** because defects were found late. This is a theoretical floor, not a realistic target.

Where the defects slipped through (phase containment)

Phase	Introduced	Carried in	Present	Detected	Escaped
Requirements	30	0	30	2 (6.7%)	28
Design	22	28	50	3 (6.0%)	47

Phase	Introduced	Carried in	Present	Detected	Escaped
Coding	38	47	85	15 (17.6%)	70
Testing	5	70	75	20 (26.7%)	55
Documentation	5	55	60	0 (0%)	60
Production	–	60	60	60	0

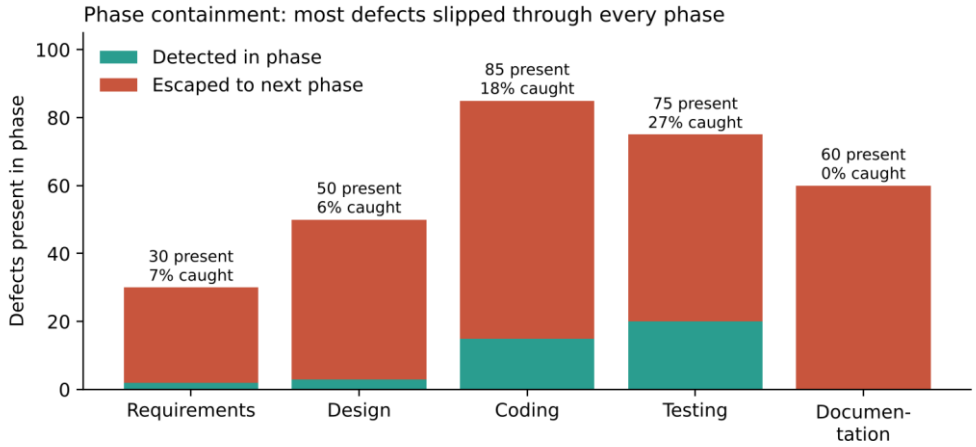


Figure 6: Phase containment: defects present vs defects caught in each phase

The requirement and design reviews (which did not exist) caught 6% of what was present. This is where the biggest improvement is available, because 28 requirement defects entered the system untouched and each of them was later carried through design, coding and testing.

3.5 Likely saving from requirement and design reviews

Method. I model the 100 defects flowing through the phases. A review in a phase removes a fraction e of the defects present in that phase at that phase's fixing cost. Defects not removed continue, and later phases keep the detection rates observed in the data (coding 17.6%, testing 26.7%, documentation 0%). Whatever remains is found in production at ₹40,000. Three scenarios were run:

Scenario (requirement / design review effectiveness)	Requirement defects caught	Design defects caught	Production defects	Total cost (₹)	Gross saving (₹)
Baseline (current data)	2	3	60	26,10,500	–
Conservative (30% / 30%)	9.0	12.9	49.8	21,81,961	4,28,539 (16%)
Moderate (50% / 50%)	15.0	18.5	42.8	18,86,621	7,23,879 (28%)
Optimistic (70% / 60%)	21.0	18.6	39.1	17,28,467	8,82,033 (34%)

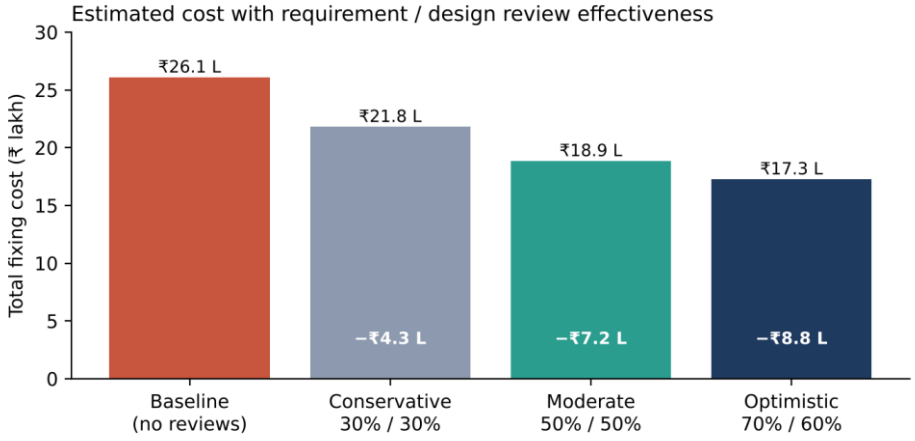


Figure 7: Estimated total cost under three review-effectiveness scenarios

Worked example (moderate scenario). Requirement review: 0.5 × 30 = 15 defects × ₹500 = ₹7,500. Design: defects present = 22 + 15 = 37; 0.5 × 37 = 18.5 × ₹1,500 = ₹27,750; 18.5 remain. Coding: 38 + 18.5 = 56.5 present; 17.6% caught = 10.0 × ₹3,000 = ₹30,000. Testing: 5 + 46.5 = 51.5 present; 26.7% caught = 13.7 × ₹8,000 = ₹1,09,600. Production: 42.8 × ₹40,000 = ₹17,12,000. Sum ≈ ₹18,86,600. Saving = 26,10,500 − 18,86,621 = **₹7,23,879**.

Net of review effort. Assuming one requirement review (5 people × 4 hours) and one design review (5 people × 6 hours) at ₹1,000 per person-hour, the effort is 20 + 30 = 50 hours = **₹50,000**. Net saving: ₹3.8 lakh (conservative), **₹6.7 lakh (moderate)**, ₹8.3 lakh (optimistic).

3.6 Limits of the estimate and its assumptions

- The effectiveness figures (30%–70%) are assumptions, not measurements of this team; real inspection effectiveness depends on reviewer skill and preparation.
- Costs per defect are averages from one release; the real cost of a production defect varies (I6, a patient-safety breach, is far costlier than I7, a manual error).
- The model counts **repair cost only**. It omits harm to patients, refunds, lost trust and legal exposure, so the true saving is larger.
- Later-phase detection rates are assumed unchanged; in practice fewer defects make testing more effective, but fixes also introduce new defects. The two effects are ignored.
- Only defects that were actually found are counted; undiscovered defects mean the true totals are higher. The ₹1,000 documentation fix cost and ₹1,000 per person-hour are my assumptions.
- The estimate is for **one release** of 100 defects and is not a forecast for later releases.

Task 4 – Apply and Critique Basic Definitions (4 marks)

Position: Careless terminology does harm quality. It hides the difference between a mistake, its consequence in the code and its visible effect, so defects are mis-classified, root causes are missed and, as incident I8 shows, wrong test results are accepted. A shared glossary is needed.

4.1 Definitions applied to MediQueue

Term	Meaning in MediQueue	MediQueue example
Error	A human mistake made by a developer, analyst or tester	A developer assumed that the fee on the booking page already included GST (I3)
Fault (defect, bug)	The incorrect item in an artefact (requirement, design, code, document) caused by an error	The payment module adds a tax amount to the displayed fee of ₹500
Failure	Observable wrong behaviour while the system runs, caused by executing a fault	A patient is shown ₹500 but charged ₹550
Test case	Inputs, preconditions, expected result and execution conditions for one test	Book an appointment for a ₹500 consultation; expected: payable amount exactly as shown on the booking page
Test oracle	The source of the expected result against which actual output is judged	The reviewed fee rule in the requirements document and the hospital tariff
Test bed	The environment (hardware, software, data, tools, SMS and payment sandboxes) in which tests run	Staging server with test patients, a payment sandbox and a stubbed SMS provider; used to run I1 and I6 with parallel requests

4.2 Error, fault and failure for I3, I5 and I6

Incident	Error (human mistake)	Fault (defect in an artefact)	Failure (observed)
I3	The team misunderstood the GST rule and did not confirm it with the finance team	The booking page shows the base fee while the payment logic adds tax (₹50 on ₹500) on top; the requirement itself recorded the wrong rule	Patient sees ₹500 on the booking page and is charged ₹550
I5	The developer treated "senior citizen" as "older than 60" or computed age by year difference or from the previous day, and nobody tested the boundary	Age comparison in the priority logic, for example age > 60, or an age computed before the birthday is counted	A patient who turns 60 on the appointment day is not given priority
I6	The developer assumed one doctor saves at a time; concurrency was not considered in design or review	Prescription PDF generation uses shared (static or global) data that is not isolated per request	Doctor B's prescription PDF shows doctor A's patient details when both save simultaneously

Note that the fault of I3 sits in the requirement, not in the code; the code does exactly what the (wrong) requirement says. A team that calls everything "a bug" cannot see this.

4.3 How confused terminology made I8 possible

In I8 a refund test was marked "Pass" because the actual output was compared with a wrong expected value. In the vocabulary above: the tester made an **error** (took a wrong expected value); this created a **fault in the test case** (wrong oracle); and the product **failure** in the refund was **never observed**, a false pass. The team used "bug" for everything, so there was no recognised category of "defect in the test", no concept of the oracle as a separate, reviewable item, and no practice of tracing an expected result to a requirement. "Pass" was read as "the software is correct", when it only meant "the software matches the number someone typed". With defined terms, a review rule such as "every expected result must cite a requirement and be reviewed by a second person" would have been natural.

4.4 Is a common glossary needed? What must it contain?

Yes. Quality problems are discussed, counted and fixed through words, and mixed words produce mixed metrics (a requirement fault logged as a "coding bug" sends the fix, the root-cause analysis and the statistics to the wrong place). The glossary should be one page, approved by the team, and kept in the repository. It must contain:

- **Error, fault/defect, failure, incident** (an incident is a reported event; a defect is its cause) and the rule that “bug” is informal and not used in reports.
- **Test case, test oracle, test bed, test suite, test log**, plus the meaning of **Pass** (actual = expected from a reviewed oracle) and **Fail**.
- **Verification and validation**, **regression testing**, **defect origin phase vs detection phase**.
- **Severity and priority scales** with definitions, and the **defect classes** used in the repository (Task 8).
- **MediQueue domain terms**: OPD working days, senior citizen, token, consultation fee vs payable amount, GST treatment, so that I2 and I3 type misunderstandings are caught at the first review.

Task 5 – Evaluate the Application of Testing Principles (3 marks)

Position: The claim "we tested everything and found no errors, so the software was correct" is wrong on at least four counts. Each is shown below with the evidence from MediQueue that violates or confirms the principle.

Principle	Evidence in the MediQueue case	Violated / confirmed
1. Testing shows the presence of defects, not their absence	A demo with "no errors" was taken as proof of correctness, yet patients found 60 defects within three months	Violated by the team; confirmed by the data
2. Exhaustive testing is impossible	"Everything" cannot be tested: 6,000 users a day, many doctors, dates, ages, name lengths and simultaneous actions. I1 and I6 occur only with same-second events	Violated: no risk-based selection of tests
3. Early testing saves time and money	No review of requirements or design; 52 of 100 defects were introduced before coding but only 5 were found there. Production fix ₹40,000 vs ₹500	Violated
4. Defect clustering	4 of the 10 incidents (I1, I2, I3, I5) sit in the booking and token flow; I1 and I6 share one cause (concurrency)	Confirmed: defects cluster in a few risky areas
5. Testing is context dependent	A healthcare application (patient identity, prescriptions, payments) needs stricter safety and privacy testing than an ordinary demo-able app (I6, I3)	Violated
6. Absence-of-errors fallacy	I2 and I3: the software did what was coded but not what the hospital needed. A system with no coding errors can still be useless or harmful if requirements are wrong	Violated
7. A test case needs a defined, reviewed expected result (oracle)	I8: a wrong expected value turned a failing refund into a "Pass"	Violated
8. Testing should not be done only by the author	Developers tested their own code, so shared misunderstanding (I3) passed unchecked	Violated

Why the developers' claim fails

1. "Found no errors" is a statement about the tests, not the software (principle 1): a weak demo finds nothing.
2. "Tested everything" is impossible (principle 2): the claim confuses effort with coverage.
3. "Software was correct" assumes the requirements were right (principle 6); I2 and I3 show they were not.
4. Even the premise is doubtful (principle 7): without a reviewed oracle, a "pass" cannot be trusted (I8).

Which principle would have had the largest effect?

Early testing (principle 3). Reasons: (a) 52% of the defects were introduced before a line of code was written, so no later testing could be as cheap; (b) in my model, reviews at requirements and design alone save ₹4.3–8.8 lakh for about ₹0.5 lakh effort (Task 3), a return of roughly 9 to 18 times; (c) it also reaches the incidents that testing code cannot catch (I2, I3, I10), and it improves the oracles used by later tests. Independent testing would save about ₹5.6 lakh but costs ₹3.2 lakh a release for a dedicated tester (my assumption), so its return is lower. The other principles matter, but they work best when applied early.

Task 6 – Evaluate the Tester's Role (3 marks)

Position: Yes, an independent tester would have changed the outcome materially: of the ten incidents, five were very likely to be caught and three more could have been caught with the tester's participation in reviews. But a tester alone would not have been enough; the value comes from the tester working inside a defined process.

6.1 Role and responsibilities across the life cycle

Phase	Tester's responsibilities
Requirements	Review requirements for clarity, testability and missing rules; ask "which days is OPD open?"; derive acceptance criteria and oracles
Design	Review design for testability, concurrency and failure handling; plan test levels and the test environment (test bed)
Coding	Support unit testing: boundary-value cases, review unit tests, help set up continuous builds
Testing	Design and execute tests; log defects in the repository with class, severity and steps; verify fixes; run regression
Release	Check exit criteria; report residual risk to management in plain terms; manage the release checklist
Maintenance	Analyse production incidents, add regression tests, feed defect statistics into process improvement

6.2 Developers testing their own code: advantages and limits

Advantages	Limitations
Deep knowledge of the code; fast feedback and cheap fixes at unit level	**Confirmation bias**: authors test to show their work is right
No hand-over delay; ideal for unit and white-box tests	**Shared misunderstanding**: the same person who misread the GST rule (I3) writes the test
Encourages quality ownership	Time pressure: testing is cut when the release date nears (as it was just before release)
	No separate view of the user, the hospital's operations or patient safety

Developer testing is necessary (it is the right place to find I4 and I5) but not sufficient. The best arrangement is developers owning unit tests and an independent tester owning higher-level validation and the oracles.

6.3 Incidents an independent tester would most likely have caught

Incident	Likelihood	Stage	Technique	Reason / limit
I3	High	System test	Compare booking page and payment amount	Internal inconsistency is visible without domain knowledge
I4	High	Unit / system test	Boundary value analysis (40, 41 characters)	Standard tester checklist item
I5	High	Unit / system test	Boundary test at age 59, 60 on the day	Standard boundary case
I7	High	Documentation / acceptance	Follow the manual step by step	Missing menu found in minutes
I10	High	System test	Load and response-time test on hospital Wi-Fi	Needs a stated target, which the tester would ask for in review
I2	Medium	Requirements review	Ask for operating-day rules and test Sundays	Only if the tester takes part in the review
I1	Medium	Integration test	Parallel booking test	Needs a tool for simultaneous requests
I6	Medium	Integration test	Concurrent save test	Intermittent; may be missed without targeted tests
I8	Low–Medium	Test review	Independent review of test cases	A tester made the error: independence is not infallibility; a second reviewer is needed

Incident	Likelihood	Stage	Technique	Reason / limit
I9	Low	Design review / monitoring	Failure injection (invalid API key)	Largely operational; needs monitoring in production

This justifies the position: 5 incidents likely and 3 possible gives roughly 6 to 8 out of 10 prevented from reaching patients. The incidents a tester would not stop alone (I8, I9) show that independence needs to be backed by review and monitoring processes.

6.4 Qualities of a tester for a healthcare application

- **Domain awareness**: understands OPD workflow, tokens, fees, priority for seniors, prescriptions and what a wrong detail means for a patient.
- **Risk and safety thinking**: puts patient safety, money and privacy first, and can explain risk to non-technical management.
- **Technical skills**: test design techniques, SQL, API and load testing, basic automation.
- **Questioning, sceptical mindset and attention to detail**: does not accept “works on my machine” or a demo as proof.
- **Communication and tact**: reports defects factually and constructively without blaming developers.
- **Ethics and confidentiality**: handles patient data responsibly, uses masked test data and follows data-protection law.

Task 7 – Trace the Origins of Defects (4 marks)

> **Position:** Half of the incidents (5 of 10) were born in requirements or design, before any code was written. MediQueue's defects mostly come from the front of the life cycle, which is the part the team did not review at all.

7.1 Origin of each incident

Incident	Origin assigned	Evidence in the case	Alternative origin and why it was not chosen
I1	**Design**	Two bookings at the same second get the same token: nothing in the design guarantees that token allocation is atomic	Coding: the fix is in code, but the missing concurrency design decision is the cause
I2	**Requirements**	"This rule was never written in the requirements"	None: the case states the origin
I3	**Requirements**	"The GST rule was misunderstood": the rule itself was captured wrongly	Coding: code only implements what was understood; the misunderstanding comes first
I4	**Coding**	Crash on names > 40 characters: missing input validation or field-length handling	Requirements: a length limit might not have been stated, but a crash is not an acceptable response to any length, so coding
I5	**Coding**	Priority missing on the exact 60th birthday: classic boundary error in age calculation	Requirements: the rule "60 or above" may be ambiguous, but the boundary should be tested whichever way it is written
I6	**Coding**	Shared data reused by two simultaneous saves: implementation defect in the PDF module	Design: if no thread-safety rule existed. Coded as the same root cause as I1 but in implementation; I chose coding because the PDF generator's data handling is internal to one module
I7	**Documentation**	Manual refers to a menu that does not exist	None
I8	**Testing**	Wrong expected value in a test case	None: a defect in the test, not in the product
I9	**Design** (trigger: environment)	Reminders stop when the provider changes its API key: the system has no failure handling or monitoring for an external dependency	Environment: the key change triggered it, but a robust design anticipates change; the defect is the missing handling
I10	**Requirements**	25-second load on hospital Wi-Fi: no performance target existed, so nobody designed or tested for one	Design: an inefficient query or payload is the technical cause, but without a requirement it cannot be called a defect to anyone

7.2 Incidents with more than one possible origin, and how I decided

I used one rule: **the origin is the earliest phase at which a correct decision would have prevented the defect.** Under this rule I1 is design (a correct design decision on concurrency prevents it for every code path), while I6 is placed in coding because the concurrency design rule would also cover it, but the specific fault is local to one module's data handling. I3 and I10 are placed in requirements because the misunderstood rule and the missing target occurred before design started. I9 is design with an environmental trigger, because external change is certain to happen and the design must expect it.

7.3 The overall pattern

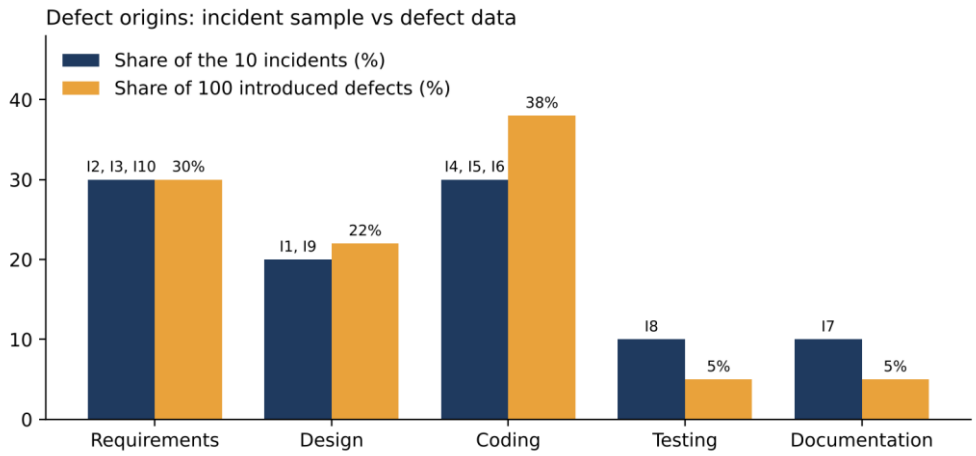


Figure 8: Origin of the ten incidents compared with the origin of the 100 defects

Origin	Incidents	Share of incidents	Defect data (introduced)
Requirements	I2, I3, I10	30%	30 of 100
Design	I1, I9	20%	22 of 100
Coding	I4, I5, I6	30%	38 of 100
Testing	I8	10%	5 of 100
Documentation	I7	10%	5 of 100
Environment	none directly	0%	–

The two pictures agree closely: requirements and design are 50% of incidents and 52% of introduced defects. Only 5 of those 52 pre-coding defects were detected before coding. **Where to invest:** (1) requirement and design reviews, where fixes cost ₹500–₹1,500 and the escape rate is highest; (2) a tester who also reviews requirements and test cases, which covers testing and documentation origins; (3) code-level controls (peer review, boundary tests) for the 38 coding defects; and (4) design for external change. This is the order used in the plan in Task 11.

Task 8 – Evaluate Defect Classes and Severity (4 marks)

Position: The classes most harmful to patients are coding or design defects that corrupt or leak patient data (I6, I1) and requirement defects that affect money or eligibility (I3, I2). The team's priorities did not reflect this: with demo-based approval, effort went to making the visible screens run, not to safety-critical logic. I recommend targeting the requirements / specification class first, after an immediate hotfix for I6.

8.1 Defect classification scheme

Class	Sub-classes	MediQueue examples
Requirements / specification	Functional description, features, feature interaction, interface description, non-functional (performance) requirements	I2 (missing rule), I3 (wrong rule), I10 (missing performance target)
Design	Algorithmic and processing, control, logic and sequence, data, module interface, external interface	I1 (concurrency control), I9 (external interface and failure handling)
Coding	Algorithmic and processing, control and logic, data and data flow, initialisation, module interface, code documentation	I4 (data / field length), I5 (algorithm, boundary), I6 (data flow, shared state)
Testing	Test harness defects, test case design and test procedure defects (including wrong oracle)	I8 (wrong expected value)
Documentation	User manual, help text, release notes	I7 (menu that does not exist)

The first four classes follow the commonly used scheme in the textbook by Burnstein; documentation is added because MediQueue has a documentation incident.

Severity and priority scales used

Severity	Meaning	Priority	Meaning
Critical	Patient-safety, privacy breach or corruption of patient data	P0	Fix immediately (hotfix within 24 hours)
High	Money, core function or data integrity wrong for many users	P1	Fix in the next release or within one week
Medium	Function impaired but workaround exists	P2	Fix in the following release
Low	Cosmetic or documentation only	P3	Backlog

8.2 Classification of the ten incidents, with reasons

ID	Class → sub-class	Severity	Priority	Justification
I1	Design → control, logic and sequence	High	P1	Two patients hold the same token: queue disputes and wrong order of care. Not life-threatening, but data integrity is lost and at 6,000 users a day same-second bookings at peak hours are certain, not rare
I2	Requirements → functional description (missing)	Medium	P2	Patients travel to a closed OPD: wasted time, inconvenient for sick patients; workaround exists (clinic cancels); fix is quick once the rule is documented
I3	Requirements → functional description (incorrect)	High	P1	Every patient is overcharged ₹50 relative to the displayed price: financial and trust impact, a legal issue, and it happens on every booking
I4	Coding → data (field length / input validation)	Medium	P2	A crash blocks booking for long names, but the patient can shorten the name; moderate share of users
I5	Coding → algorithm and processing (boundary)	Medium	P3	A narrow group (turned 60 on the day) loses priority: unfair but low frequency and no direct harm
I6	Coding → data flow / shared state	Critical	P0	A prescription with another patient's details is a **patient-safety and privacy failure**; it can lead to wrong treatment. Fix immediately, regardless of frequency
I7	Documentation → user manual	Low	P2	No harm, but a fix is a one-line change and it generates support calls, so it is cheap to fix early
I8	Testing → test case design (wrong oracle)	High	P2	Hides a real refund failure, a money defect that no one knows is present: latent risk. Fix by correcting the oracle and re-running

ID	Class → sub-class	Severity	Priority	Justification
I9	Design → external interface description	High	P2	Reminders stop silently so patients miss appointments; a monitoring gap makes it easy to repeat
I10	Requirements → non-functional (performance)	Medium	P2	25 seconds slows doctors' work and queues; no data loss; fix after a target is agreed



Figure 9: Severity vs priority of the ten incidents

8.3 Why severity and priority can differ

Severity is the impact if the defect occurs (how bad). **Priority** is the order in which the team fixes it, and also depends on how often it occurs, whether a workaround exists, the cost of the fix, deadlines and regulation.

- **I7** has low severity (no harm to anyone) but I assigned medium priority (P2): the fix is trivial, it affects every patient who reads the manual and it creates avoidable support calls.
- **I1** has high severity and is high priority (P1) here only because the hospital's volume makes collisions routine. A similar defect in a clinic with 20 bookings a day might be high severity but lower priority. This is the usual reason for the two scales to diverge.
- **I6** is a case where priority follows severity completely: a patient-safety defect is P0 whatever its frequency.

8.4 Which defect class to target first

Requirements / specification defects, first. (1) 30 of 100 defects were introduced here and 28 escaped, the worst containment of any phase (6.7%); (2) they are cheapest to fix (₹500); (3) they are the root of financial and eligibility harm (I2, I3, I10); (4) they contaminate everything downstream, since design, code and the tester's oracles are derived from them. **Coding defects** are the largest class by count (38) and are second priority, with an **immediate hotfix for I6** because it is a patient-safety failure. The priorities of the team (own-code testing, demo approval) targeted the opposite: they spent effort on coding (15 detections) while requirements (2) and design (3) were ignored.

Task 9 – Design and Appraise a Defect Repository (3 marks)

Position: A defect repository is the team's memory. It must record not only what failed but also where the defect was introduced, where it was found and why, so the team can count, compare and prevent. The current e-mails and spreadsheet cannot do this.

9.1 Fields of the MediQueue defect repository and why each is needed

Field	Why it is needed	Example (incident I6)
Defect ID (unique)	Single reference across code, tests and reports; avoids duplicate handling	MQ-0006
Title and description	Shared understanding of the problem in one line and in detail	Prescription PDF shows previous patient's details
Steps to reproduce	Lets the developer reproduce; makes the defect testable and repeatable	Two doctors save a prescription within the same second
Expected vs actual result	Records the oracle and the failure; shows I8-type wrong oracles	Expected: own patient's data; actual: other patient's data
Environment / build / version	Separates product defects from environment defects; links to release	Production, web v1.4.2, Chrome
Module / component	Finds defect clusters (principle 4)	Prescription service
Reported by, date, source	Accountability; shows how found (patient, review, test) and how many were found internally	Patient complaint via helpdesk, 12-Mar
Defect class and sub-class	Enables analysis by class (Task 8)	Coding → data flow / shared state
Origin phase (injected)	Needed to compute where defects come from (Task 7)	Coding
Detected phase	Compared with origin phase gives phase containment and cost of lateness (Task 3)	Production
Severity and priority	Triage and scheduling; separate impact from urgency	Critical / P0
Patient-safety / privacy flag	Marks defects that may need regulatory action; becomes essential in Task 12	Yes
Status and workflow history	Tracks progress and keeps an audit trail (see Figure 10)	Triaged → Assigned → Fixed → Verified → Closed
Assigned to, target release	Ownership and planning	Developer A, hotfix 1.4.3
Root cause category	Drives prevention (e.g. missing review, missing concurrency design, wrong oracle)	Shared mutable state; no thread-safety checklist
Fix description and link to change	Traceability to code; supports review of the fix	Commit reference
Linked requirement and test case	Traceability: which requirement or test should have caught it; supports regression	Requirement R-31; new concurrent-save test T-118
Fix effort and cost	Builds real cost-per-phase data instead of assumptions	6 person-hours
Verification result and regression test added	Confirms the fix and prevents recurrence	Verified by tester; regression test added: Yes
Duplicate / related defects	Detects repeats (I1 and I6 share a root cause)	Related: MQ-0001

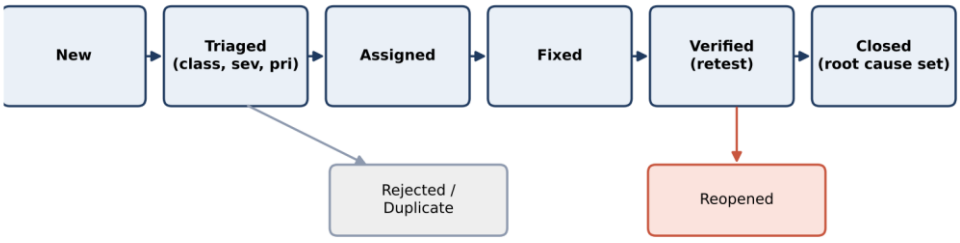


Figure 10: Defect life cycle used in the repository

9.2 How the repository supports analysis, improvement and prevention

Purpose	Typical question and report	Decision it supports
Defect analysis	Pareto chart by class and module; origin vs detection phase matrix	Shows where defects come from and cluster (Tasks 7 and 8)
Process improvement	Phase containment and defect removal efficiency per release; cost per defect by phase	Judges whether reviews and the tester are paying for themselves (Task 3 and 11)
Prevention	Root-cause categories; recurrence rate; defects with no regression test	Adds checklists, coding rules and tests to stop repeats
Release decision	Open Critical / High defects; trend of new defects	Evidence for the release gate instead of “the demo went well”

9.3 Weaknesses of the current e-mail and spreadsheet approach

Weakness	Consequence for MediQueue
No unique ID, duplicates not detected	The same complaint appears many times; defect counts are meaningless
Free text, no classification	Cannot count by class, origin or module; no learning
No workflow or history	Nobody knows who owns a defect or whether it was verified; defects are lost
No link to requirements, tests or code	Cannot see why a defect escaped or whether a regression test exists
No reproduction or environment data	Developers cannot reproduce intermittent defects such as I1 and I6
Version conflicts and no audit trail	Spreadsheets are overwritten; no proof of what was done, which matters for health-data incidents
Weak access control, and patient details in e-mail	Patient data circulates in inboxes: a privacy risk. The repository should store masked patient data only
No metrics	Management cannot see trends; “quality” stays a feeling

A free or low-cost tracker (for example Jira, Bugzilla or GitHub Issues) with the fields above and mandatory fields for class, origin phase and severity is sufficient for a five-person team; the discipline of filling the fields matters more than the tool.

Task 10 – Evaluate Defect Examples and Prevention Strategies (4 marks)

Position: I selected I3 (requirements), I6 (coding) and I8 (testing). For each, one process change would have prevented it. The best value comes from the upstream review: for about ₹20,000 per cycle it addresses I3 and also I2 and I10, returning about 16 times its cost.

10.1 Root-cause analysis

Incident	Class	Root cause (why-why chain)	Root cause statement
I3	Requirements	Why was ₹550 charged? Payment adds tax. Why? Rule understood differently from the page. Why? Rule captured from a conversation, not confirmed. Why? No requirement review, no finance sign-off	Pricing rule never baselined or reviewed with the owner
I6	Coding	Why wrong details? Shared data used by two saves. Why? Code assumed one user at a time. Why? Not considered. Why? No code review, no concurrency checklist, no parallel test	No review or test of shared state
I8	Testing	Why "Pass"? Compared with wrong value. Why? Expected value typed by tester from memory. Why? No documented oracle or second review	No reviewed oracle linked to a requirement

10.2 Prevention strategy, cost and benefit

Benefits come from the Task 3 model (each strategy tested alone); effort cost uses ₹1,000 per person-hour. All are estimates.

Incident	Prevention strategy	Effort (cost)	Estimated benefit	Benefit ÷ cost	Other defects covered
I3	Formal **requirement inspection** with finance and OPD staff; pricing rules in a table signed by the owner	5 people × 4 h = ₹20,000 per cycle	₹3,17,805 (50% effectiveness)	≈ **16**	I2, I10
I6	**Peer code review** with a shared-state and thread-safety checklist, static analysis, a parallel-save unit test	About 1 h per developer per week over 32 weeks = 160 h = ₹1,60,000	₹5,40,867 (coding-phase detection 17.6% → 40%)	≈ **3.4**	I1, I4, I5
I8	**Independent review of test cases**: every expected result cites a requirement and is checked by a second person	About 16 person-hours = ₹16,000 per release	About ₹1,20,000 (3 of 5 testing-origin defects no longer mask a ₹40,000 production defect)	≈ **7.5**	Any wrong oracle

For reference, a design review (5 people × 6 h = ₹30,000) saves ₹5,48,749 alone (about 18 times), and the requirement plus design reviews together save ₹7,23,879 for ₹50,000 (about 14.5 times).

10.3 Which strategy gives the best value

Reviews of requirements and design: they have the highest benefit-to-cost ratio (about 14–18) because (a) the defects are fixed at ₹500–₹1,500 rather than ₹40,000, (b) one review sees many defects of many kinds, and (c) their effort is small and does not need new staff. The test-case review (≈7.5) is cheap and needed. Peer code review has the lowest ratio (≈3.4) but it is the only direct control over coding defects, which are 38% of the total, and remains justified by I6's severity. Value is a balance: reviews first, because they are cheapest, then the others.

Task 11 – Recommend and Defend a Quality Improvement Plan (4 marks)

Position: Introduce a small, ordered set of changes, each tied to an incident or a defect class found in Tasks 1–10. Expected result: production defects fall from 60 to about 24 and fixing cost from ₹26.1 lakh to ₹12.4 lakh per release, at about ₹5.5 lakh of added effort (net benefit about ₹8.2 lakh).

11.1 Process changes and their order of priority

Order	Change	Evidence that drives it	When	Effort
1	**Hotfix** I6, I3, I1; start the defect repository	I6 is Critical; I3, I1 High; no history of defects (Task 9)	Weeks 0–2	Fix effort; ₹16,000 for set-up (2 days)
2	**Glossary, test policy and oracle rule** (every expected result is reviewed and cites a requirement)	I8; terminology (Task 4)	Weeks 0–3	Few hours
3	**Requirement and design reviews** with hospital staff	Requirements are the first class to target; ₹7.2 lakh saving (Task 3)	From week 2, every milestone	₹50,000 per cycle
4	**Independent tester** joining reviews, designing tests, owning the oracle	Task 6: 5 incidents likely and 3 possible to catch	From week 3	About ₹3.2 lakh for 8 months (assumed ₹40,000 a month)
5	**Peer code review** with checklist and static analysis	Coding is 38% of defects; I4, I5, I6	From week 4	₹1.6 lakh
6	**Automated regression** for booking, token and payment; monitoring for SMS	I1, I3, I9, clustering in booking	Weeks 8–16	Part of tester and developer time

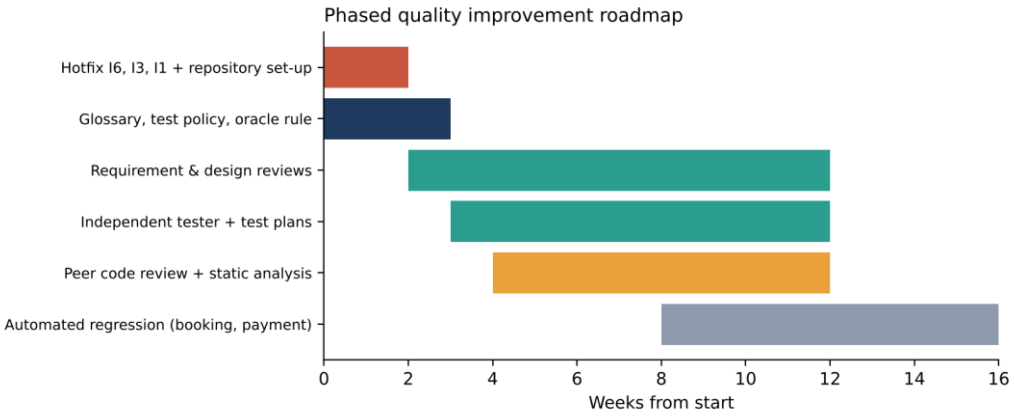


Figure 11: Phased roadmap of the quality improvement plan

11.2 Roles of testers, reviews and the defect repository

- **Testers:** own the test plan, oracles, system-level tests and the release exit checklist; join requirement and design reviews; verify every fix.
- **Reviews:** the main preventive control, covering requirements, design, selected code and test cases, each with a short checklist.
- **Defect repository:** the measuring instrument; after each release we compute defect removal efficiency, phase containment and cost per defect and review them in a retrospective.

11.3 Expected effect on number and cost of defects

Plan assumptions: requirement review 50%, design review 50%, coding-phase detection 40% (up from 17.6%) and testing-phase detection 50% (up from 26.7%) of the defects present. Running the Task 3 model gives:

Measure	Baseline	With plan	Change
Detected before coding (req. + design)	5	33.5	+28.5
Detected in coding	15	22.6	+7.6
Detected in testing	20	19.4	–

Measure	Baseline	With plan	Change
Reach production	60	24.4	−59%
Total fixing cost	₹26,10,500	₹12,36,650	−₹13,73,850 (53%)
Added effort (reviews, tester, code review, repository)	–	₹5,50,000	–
Net benefit per release	–	**₹8,23,850**	Return on effort ≈ 2.5 ×

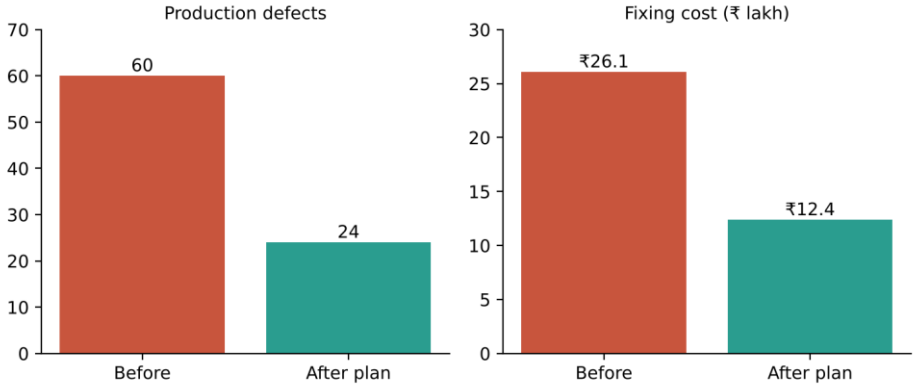


Figure 12: Expected effect of the plan on production defects and cost

11.4 Trade-offs, risks and limitations

Risk / trade-off	Why it matters	Mitigation
Added effort of 8–10% of developer time	Slows feature delivery in the short term	Short, checklist-driven reviews; track the saving to justify them
Estimates are assumptions (effectiveness, ₹ per hour, tester salary)	The numbers may be wrong by a large factor	Measure real values in the repository and update the model after each release
Tester becomes the "safety net" and developers stop caring	Quality ownership falls	Developers keep owning unit tests; defects are reported by origin, not by blame
Review fatigue or reviews done carelessly	Lower effectiveness than assumed	Time-box (60–90 minutes), rotate reviewers, use checklists, measure defects found per review hour
Repair-cost model omits harm and reputation	True benefit is larger, but uncounted	Report patient-safety defects separately
Dependence on hospital staff for requirement review	Reviews cannot happen without them	Book review slots in advance; use a short scenario-based format

11.5 Is it practical for five developers?

Yes. The plan adds one dedicated tester and an estimated 8–10% of developer time: peer code review about 1 hour a week per developer, requirement and design reviews of 4 to 6 hours at each milestone for five people, and about an hour a week for the repository and glossary. It uses free tools (a tracker, static analysers) and no new development framework. The one structural change is the tester; the rest is discipline. If a tester cannot be hired at once, a developer from another module can act as reviewer for test cases and requirements on rotation as a temporary measure, accepting reduced independence.

Task 12 – Day-4 Design Challenge: Re-evaluate and Adapt the Quality Process (8 marks)

> **Decision: MODIFY.** The foundations of the original plan (defined process, independent tester, reviews, a defect repository with classification) remain valid because the defect data have not changed. But the timing, depth and automation of each activity must change for a two-week release cycle, and a new class of external-system defects and a higher severity for data-sharing must be added.

A. Impact Analysis (2 marks)

A1. What has changed

Dimension	Original situation	New situation
Release frequency	About every six months (large releases)	Every two weeks: about 26 releases a year, 13 times as many; each has about 10 working days
Dependencies	Self-contained system; the SMS provider is the only external service	National digital health-ID system maintained by an external agency; its behaviour, versions and downtime are not under our control
Data scope	Records stay inside the hospital	Patient health records are shared between hospitals
Impact of a defect	Wrong detail shown to one patient at one hospital	A wrong or leaked record can propagate to other hospitals and is a regulatory (data-protection) matter

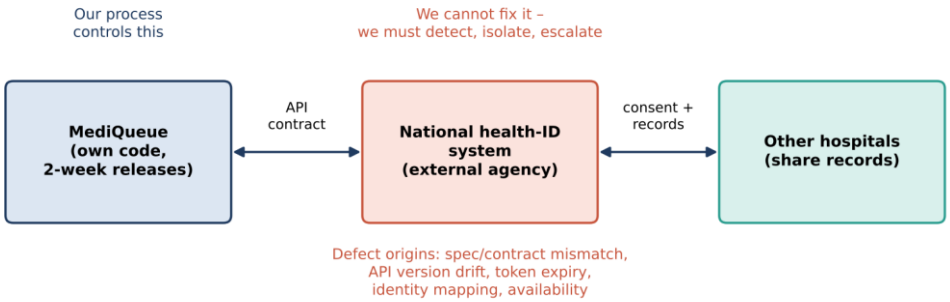


Figure 13: MediQueue, the external health-ID system and other hospitals

A2. Assumptions of my plan that are affected

Assumption in the Task 11 plan	Still valid?	Why
Reviews and test phases are scheduled as separate phases of a large release	**No**	A two-week cycle has no room for week-long phases; they must fit inside the sprint
Regression can be done manually at the end	**No**	26 manual full regressions a year are not affordable; the regression pack must be automated
All defects are found in our own code	**No**	Defects can come from the external agency's API, versions, data and downtime
Defect repository fields are sufficient	**Partly**	Needs fields for external systems, consent and patient-impact (see section B)
Severity scale as in Task 8	**Partly**	Severity of identity, access and data-sharing defects rises (A4)
Independent tester works at the end of the project	**No**	The tester must work inside each sprint
Defect data (requirements and design first) and cost ratios	**Yes**	Still valid: early detection remains the cheapest; the escalation (80×) is the reason to shift left even more strongly

A3. Why frequent releases change the role of testing and the tester

With six-month releases, testing is a phase at the end and the tester acts as a gatekeeper who finds defects before release. With a release every two weeks there is no "end": every sprint contains planning, development,

testing and release, and every release carries regression risk from every earlier release. Therefore: (1) testing shifts left into the sprint (acceptance criteria and test ideas at planning, not after coding); (2) regression must be automated and run on every change, otherwise the test effort grows with every release; (3) tests are chosen by **risk** because there is no time to test everything (principle 2); (4) the tester becomes an **embedded quality partner** who reviews stories, automates checks, explores risky areas, maintains the release gate and watches production, instead of only executing tests; and (5) feedback from the repository is used every sprint, not once a year.

A4. Why sharing health records raises the severity of some defect classes

Defect class (example)	Severity before	Severity after sharing	Reason
Coding: wrong patient's data (I6-type)	Critical	**Critical + reportable**	The wrong record can leave the hospital; it is a data-protection breach and can affect treatment elsewhere
Requirements: identity / consent rules	High	**Critical**	Mis-matching patients or ignoring consent exposes or corrupts another person's record
Design: access control and data mapping	High	**Critical**	Over-sharing or mapping errors reach other hospitals
Design: external interface (I9-type)	High	**High–Critical**	A missed API change or expired token blocks record access during care
Coding: validation (I4-type, long names)	Medium	**High**	Truncated or rejected names break patient identity matching with the ID system
Documentation (I7-type)	Low	**Medium**	Consent notices and patient instructions become legally relevant

B. Evaluation of the Original Plan (2 marks)

B1. Do the planned reviews and test activities fit into a two-week cycle?

Planned activity	Fits as is?	Adaptation
Formal requirement and design inspection (several people, hours of preparation)	**No**	Use a 30–45 minute story-level review at sprint planning with the tester present (three-perspective review); keep formal inspections only for high-risk changes: consent, identity matching, data sharing and payments
Peer code review with checklist	**Yes**	Keep it as a pull-request review with the checklist; it fits the sprint naturally
Independent tester in the later phase	**No**	Tester embedded in the sprint, testing stories as they complete
Manual regression	**No**	Automate regression for booking, token, payment and record sharing; run on every build
Release approval by management	**No**	Replace by an exit checklist of automated results and open Critical/High defects, decided by the team lead and tester
Retrospective from the repository	**Yes**	Run after every release, with fewer, targeted metrics

B2. New defect origins introduced by the external health-ID system

- **Contract or specification mismatch:** our reading of the agency's interface differs from the real one (similar in nature to I3).
- **Interface misuse in our code:** wrong fields, formats, encoding or error handling.
- **Provider change:** new API version, key or token rotation (the same pattern as I9, now with much higher stakes).
- **Provider defect:** wrong or inconsistent responses from the national system.
- **Availability and performance:** downtime or slow responses (the same pattern as I10), which now block care.
- **Identity and data quality:** duplicate or mismatched health IDs, name and date-of-birth differences, incomplete records.
- **Environment difference:** the test sandbox differs from production behaviour.

- **Consent and privacy errors:** sharing a record without valid patient consent.

B3. Is the defect repository still sufficient?

No. It was designed for defects in our own artefacts. The following fields must be added:

New field	Purpose
External system flag and name	Separates defects involving the health-ID system or other hospitals
API / contract version and endpoint	Finds version drift and contract mismatches
Defect ownership (us / agency / shared / unknown)	Decides who must fix it and who to escalate to
Partner ticket ID, escalation date, SLA clock	Tracks the dependency until closed
Workaround and patient impact count	Shows how many patients are affected while waiting for the partner
Patient data / consent flag and regulatory-reportable flag	Triggers the privacy-incident process and legal timelines
Detected in (pipeline stage: story review, CI, staging, production)	Measures shift-left effectiveness every sprint
Release number and time to fix (hours)	Supports two-week trend reporting

C. Adaptation or Redesign (2 marks)

C1. Decision: MODIFY

Option	For	Against	Verdict
Retain	Plan is based on real defect data; cheap	Cannot work in two-week cycles; ignores external dependency and data-sharing risk	Rejected
Modify	Keeps proven foundations (reviews, independent tester, repository, classification); adds automation, contract testing and external-defect tracking; practical for five developers	Needs some new skills (automation, contract tests) and discipline	**Chosen**
Redesign	Fresh process can be designed for continuous delivery	The defect data still show that requirement and design defects dominate; throwing away the plan would lose its basis; too much change for a small team in a short time	Rejected

C2. Revised process and the tester's revised role

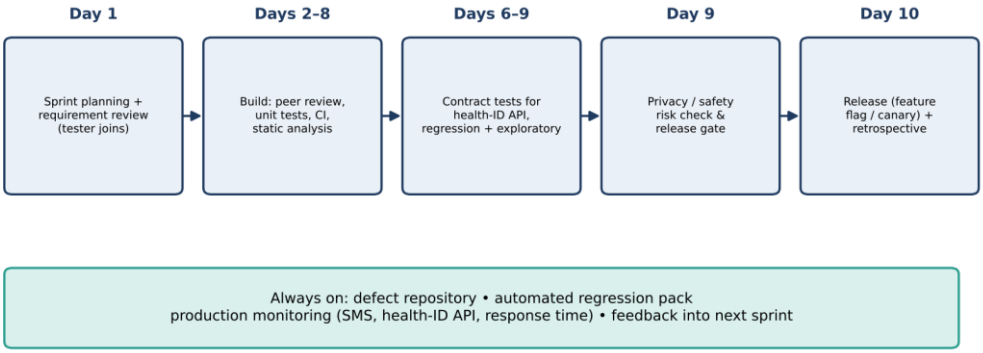


Figure 14: Revised two-week quality process

- **Sprint planning (day 1):** story review with the tester; acceptance criteria and oracles written; risks marked (consent, identity, payment, external API).
- **Build (days 2–8):** peer code review with checklist, unit tests with boundary values, static analysis and continuous integration on every commit.
- **Verify (days 6–9):** automated regression; **contract tests** against the health-ID API (and a stub for downtime and slow response); exploratory testing on risky stories; load check on the doctor's schedule screen.

- **Release gate (day 9):** privacy and safety check on anything touching shared records; zero open Critical/High defects; rollback plan.
- **Release (day 10):** staged release behind a feature flag; monitoring for SMS, API errors and response time; retrospective with repository metrics.

Tester's revised role: from end-of-project gatekeeper to embedded quality partner who (1) reviews stories for testability and risk, (2) designs and automates the regression and contract tests, (3) explores new features, (4) owns the release checklist and production monitoring, and (5) maintains the repository and its metrics. In a team of five developers, one tester is the minimum; a second part-time tester or a developer with automation skills should be added for the integration work.

C3. Testing principles that now matter more

Principle	Why it is more important now
Early testing	There are only 10 days per release: defects found after coding cost far more than the 80× ratio suggests, given the release pressure
Exhaustive testing is impossible	Risk-based selection of tests is necessary: concentrate on consent, identity, payments and external-API flows
Pesticide paradox	Repeated identical regression tests stop finding new defects; the suite and exploratory sessions must be refreshed every sprint
Defect clustering	Integration points with the external system are the new hot-spot
Context dependence	Health data and patient safety demand stricter gates than ordinary application releases
Absence-of-errors fallacy	Passing contract tests does not prove that the integration meets clinical and legal needs; validate with hospital staff and the agency's sandbox

C4. Classifying and tracking defects from the external system

Code	Category	Owner	Example	Tracking action
E1	Contract / specification mismatch	Us	We misread the field format of the ID response (I3-type)	Fix in our code; correct the contract notes; add a contract test
E2	Interface misuse	Us	Our request omits a required field	Fix; add a unit and contract test
E3	Provider defect	Agency	The ID service returns a wrong record	Log with partner ticket ID and SLA clock; implement a workaround; escalate
E4	Provider change (version, key, token)	Shared	Key rotation stops lookups (I9-type)	Monitor and alert; version-pin; agree a change-notice process
E5	Availability / performance	Agency	Slow response blocks patient check-in (I10-type)	Time-outs, fallback behaviour; track against the SLA
E6	Identity / data quality	Shared	Duplicate health IDs; name mismatch	Matching rules; data-quality report to the agency

Every external defect also records the repository fields from B3. Any defect that exposes one patient's data to another, regardless of owner, is automatically **Critical**, flagged as reportable, and handled through the privacy-incident process.

D. Evidence-Based Defence (2 marks)

Evidence	What it shows	Why it supports MODIFY
Defect data and cost	Production defects are 60% of defects but 91.9% of cost (₹24 of ₹26.1 lakh); a production fix costs 80× a requirements fix	The case for early, cheap detection is stronger when releases are frequent; the original plan's basis remains valid, so retain it
Origins and classes	52% of defects are introduced before coding; 5 of 10 incidents come from requirements or design; requirement defects escape most (6.7% caught)	Keep requirement and design review; make it lightweight to fit the sprint
Release frequency	One release carried 100 defects over about 8 months, roughly 6 per fortnight. If the old escape rate stayed, about 3.5 would reach production every release (₹1.4 lakh a fortnight, ₹36.7 lakh a year over 26 releases). With the plan's 24% escape, about 1.4 per release, ₹0.57 lakh a fortnight, ₹14.9 lakh a year, saving about ₹21.8 lakh	Automation and in-sprint testing are needed; the saving is much greater than the tester's annual cost of about ₹4.8 lakh (assumption). Small, frequent releases also reduce the blast radius of each one

Evidence	What it shows	Why it supports MODIFY
Patient-safety and data-privacy risk	I6 shows what a data-mix-up costs; sharing records across hospitals turns it into a reportable breach	Raise severity; add a privacy gate, the consent flag and a Critical rule for any cross-patient exposure
Team size and capacity	5 developers × 10 days = 50 developer-days per sprint. Quality activities in the plan take about 8–10% of developer time (about 4–5 days) plus one tester	Practical: no new large team. Redesign would exceed capacity, and retaining the old plan would not fit the cycle
External dependency	I9 and I10 already show that external and environmental behaviour causes defects	E1–E6 classes, contract tests and monitoring address that without a full redesign

Conclusion. The evidence shows the original plan was right about where defects come from and how much late detection costs; what has changed is the speed and the risk. The right response is therefore to **modify, not retain or redesign**: keep the preventive reviews, independent tester, repository and classification; make reviews story-sized; automate regression; add contract tests and external-defect tracking; and raise severity and add a privacy gate for shared health records. Estimated results are illustrative and should be replaced with measured repository data after the first few sprints.

Appendix A – Assumptions, Limitations and References

A.1 Assumptions used in the calculations

Assumption	Where used
Fixing cost of a documentation defect = ₹1,000 (not given in the case)	Task 3 ideal-cost calculation, scenario model
Review effectiveness 30–70% of the defects present in requirements and design	Task 3 scenarios; Tasks 10, 11, 12
Later phases keep their observed detection rates (coding 17.6%, testing 26.7%, documentation 0%)	Task 3 model
Blended effort cost = ₹1,000 per person-hour	Review and code-review cost (Tasks 3, 10, 11)
Tester cost = ₹40,000 a month; 8 months per release project	Task 11 cost; Task 12 annual cost (₹4.8 lakh)
Code review effort = 1 hour per developer per week for 32 weeks	Task 10 and 11
Plan effectiveness: coding detection 40%, testing detection 50%	Task 11 expected effect
Defect injection rate constant: 100 defects per about 8 months (≈ 6 per fortnight)	Task 12 D, release-frequency comparison; actual injection per sprint will be lower for small changes

A.2 Limitations

- All savings are based on one release's defect data and the repair-cost figures in the case; they are estimates for decision support, not forecasts.
- The models count only repair cost. Patient harm, privacy breach penalties and reputation loss are not priced.
- Known incident descriptions are short; some causes in Tasks 4 and 7 (for example the exact fault in I3 or I5) are reasoned inferences and should be confirmed by code inspection.

A.3 References

- Burnstein, I. (2003). Practical Software Testing: A Process-Oriented Approach. Springer. (Testing as an engineering activity, TMM, testing principles, defect classes and repository.)
- Myers, G. J., Sandler, C. and Badgett, T. (2011). The Art of Software Testing (3rd ed.). Wiley.
- ISTQB. Certified Tester Foundation Level Syllabus: seven testing principles and terminology.
- CSA37 Software Testing, Assignment 1 brief: MediQueue case, incident list and defect data table.